
1. 建文件夹 搭venv环境

```
python3.10 -m venv .venv  
source .venv/bin/activate
```

2. 装基础 TensorFlow, 再装 tensorflow-metal

问题: tensorflow 和 tensorflow-metal 版本不兼容

解决方案: 放弃 tensorflow-metal, 直接用 CPU 跑

3. 下载Kaggle 的 PlantVillage 数据集, 选择类别 (全选的话跑的太慢)

```
Pepper__bell__Bacterial_spot  
Pepper__bell__healthy  
Potato__Early_blight  
Potato__Late_blight  
Potato__healthy  
Tomato__Early_blight  
Tomato__Late_blight  
Tomato__healthy
```

4. 跑train.py

```
import os  
import tensorflow as tf  
import matplotlib.pyplot as plt
```

```
DATA_DIR = "data"  
IMG_SIZE = (224, 224)  
BATCH_SIZE = 32  
EPOCHS = 5  
SEED = 123
```

```
train_ds = tf.keras.utils.image_dataset_from_directory(  
    DATA_DIR,  
    validation_split=0.2,  
    subset="training",  
    seed=SEED,  
    image_size=IMG_SIZE,
```

```

        batch_size=BATCH_SIZE,
    )

val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
)

class_names = train_ds.class_names
num_classes = len(class_names)

print("Classes:", class_names)

AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)

data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.1),
])

model = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0 / 255, input_shape=(224, 224, 3)),
    data_augmentation,

    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(64, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(128, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(num_classes, activation="softmax"),
])

model.compile(

```

```

optimizer="adam",
loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)

os.makedirs("outputs/figures", exist_ok=True)

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="train_accuracy")
plt.plot(history.history["val_accuracy"], label="val_accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Training vs Validation Accuracy")
plt.savefig("outputs/figures/baseline_accuracy.png")
plt.close()

plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="train_loss")
plt.plot(history.history["val_loss"], label="val_loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.title("Training vs Validation Loss")
plt.savefig("outputs/figures/baseline_loss.png")
plt.close()

model.save("outputs/checkpoints/baseline_cnn.keras")

print("Training finished.")
print("Model saved to outputs/checkpoints/baseline_cnn.keras")

```

5. 第一次结果

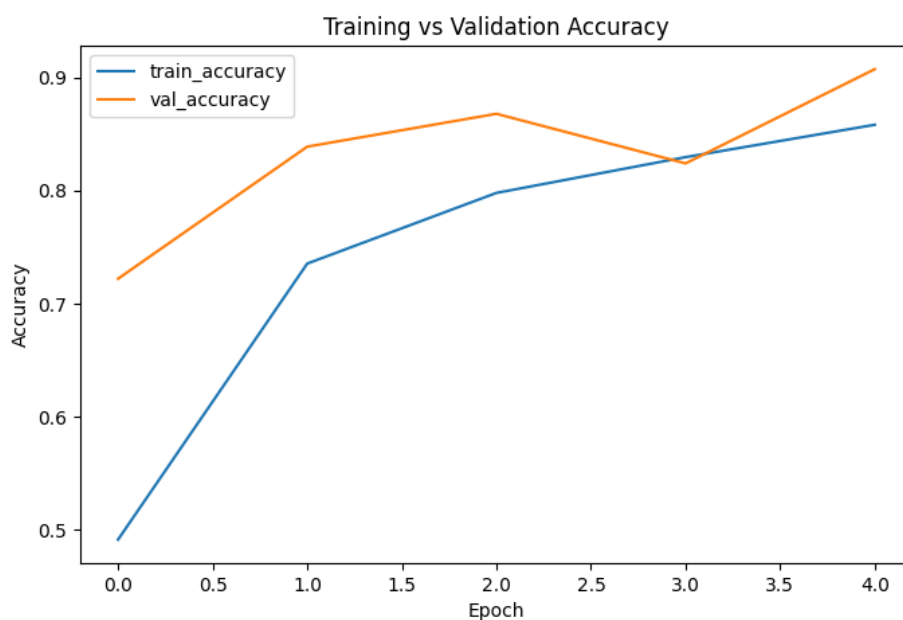
Epoch 1/5

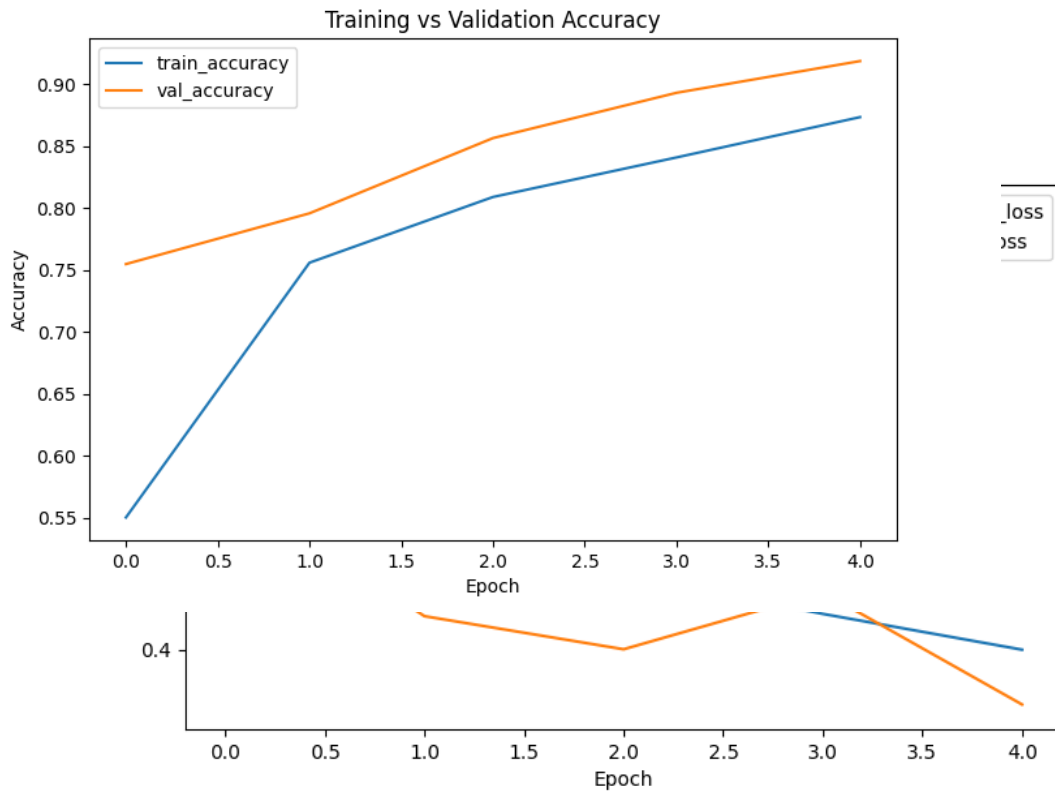
82s 353ms/step - accuracy: 0.4916 - loss: 1.3988 - val_accuracy: 0.7222 - val_loss: 0.7802
Epoch 2/5
90s 391ms/step - accuracy: 0.7356 - loss: 0.7465 - val_accuracy: 0.8389 - val_loss: 0.4766
Epoch 3/5
87s 381ms/step - accuracy: 0.7980 - loss: 0.5722 - val_accuracy: 0.8679 - val_loss: 0.4015
Epoch 4/5
91s 395ms/step - accuracy: 0.8296 - loss: 0.4818 - val_accuracy: 0.8241 - val_loss: 0.5310
Epoch 5/5
94s 411ms/step - accuracy: 0.8583 - loss: 0.4006 - val_accuracy: 0.9074 - val_loss: 0.2763

$$\text{accuracy} = \frac{\text{预测正确的样本数}}{\text{总样本数}}$$

$$\text{loss} = -\log(p_{\text{true class}})$$

你给正确答案的概率越低 → 惩罚越大 (loss)





6. 第二次实

验

改变：把数据分成70% train 15% val 15% test

结果：

Epoch 1/5

73s 356ms/step - accuracy: 0.5501 - loss: 1.2819 - val_accuracy: 0.7548 - val_loss: 0.6976

Epoch 2/5

73s 363ms/step - accuracy: 0.7559 - loss: 0.6902 - val_accuracy: 0.7958 - val_loss: 0.5931

Epoch 3/5

74s 368ms/step - accuracy: 0.8090 - loss: 0.5443 - val_accuracy: 0.8565 - val_loss: 0.4337

Epoch 4/5

76s 380ms/step - accuracy: 0.8409 - loss: 0.4431 - val_accuracy: 0.8931 - val_loss: 0.3335

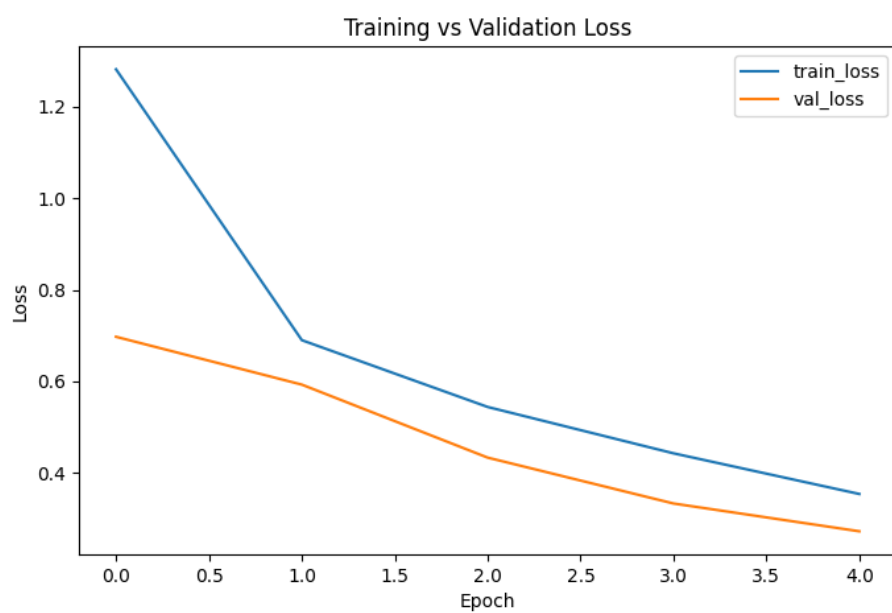
Epoch 5/5

81s 406ms/step - accuracy: 0.8735 - loss: 0.3542 - val_accuracy: 0.9187 - val_loss: 0.2727

5s 110ms/step - accuracy: 0.9047 - loss: 0.3183

Test Loss: 0.3183

Test Accuracy: 0.9047



7. 对 test images 人工加扰动（增强版evaluation）

干扰	模拟什么问题	测试什么能力
Blur	对焦差 / 手抖	纹理依赖
Brightness	光照变化	光照鲁棒性
Rotation	拍摄角度	方向不变性

Crop	拍不全	空间完整性依赖
Noise	噪声干扰	抗噪能力
Clutter	背景复杂	注意力是否集中

===== Summary =====

clean -> loss=0.3183, accuracy=0.9047
blur -> loss=1.0565, accuracy=0.6880
brightness -> loss=0.6143, accuracy=0.8444
rotation -> loss=0.6143, accuracy=0.8022
crop -> loss=0.9624, accuracy=0.7127
noise -> loss=0.3493, accuracy=0.8982
clutter -> loss=2.1634, accuracy=0.6364

The model is robust to noise and relatively stable under brightness changes, but its performance drops significantly under blur, crop, and background clutter. This suggests that the model relies heavily on fine-grained texture (blur的下降) and complete spatial information (crop的下降), and is sensitive to visual degradation and irrelevant background features (clutter的严重下降).

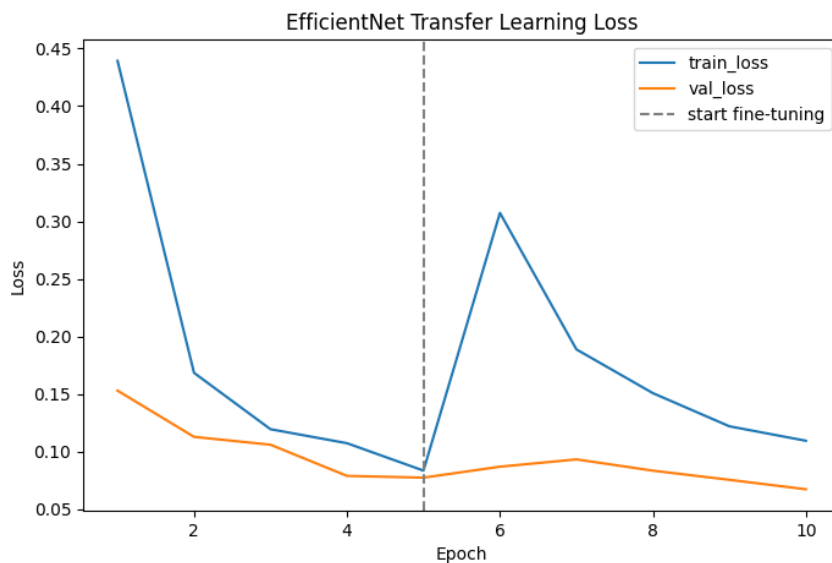
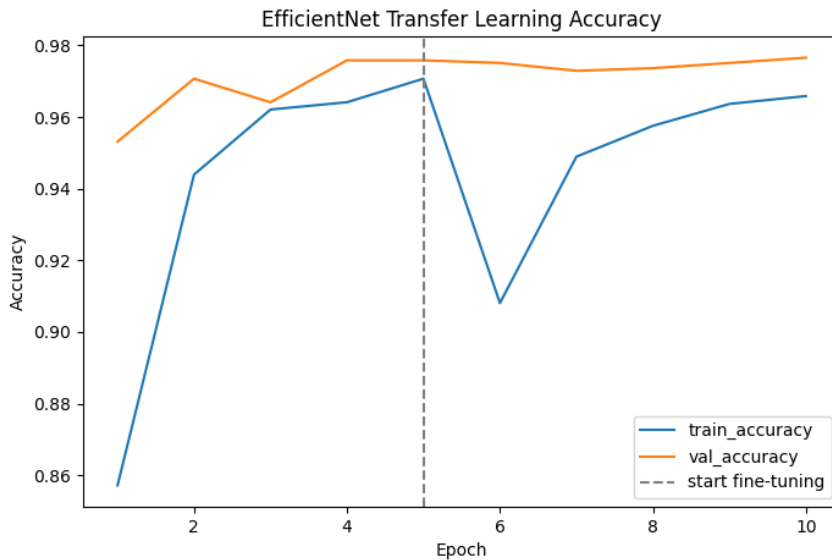
8. Transfer Learning Model

===== Stage 1: Train classification head =====

Epoch 1/5
60s 284ms/step - accuracy: 0.8572 - loss: 0.4393 - val_accuracy: 0.9531 - val_loss: 0.1530
Epoch 2/5
67s 337ms/step - accuracy: 0.9439 - loss: 0.1684 - val_accuracy: 0.9707 - val_loss: 0.1128
Epoch 3/5
94s 470ms/step - accuracy: 0.9621 - loss: 0.1195 - val_accuracy: 0.9641 - val_loss: 0.1060
Epoch 4/5
92s 459ms/step - accuracy: 0.9641 - loss: 0.1073 - val_accuracy: 0.9758 - val_loss: 0.0789
Epoch 5/5
97s 488ms/step - accuracy: 0.9707 - loss: 0.0835 - val_accuracy: 0.9758 - val_loss: 0.0774

===== Stage 2: Fine-tuning =====

Epoch 6/10
125s 606ms/step - accuracy: 0.9081 - loss: 0.3073 - val_accuracy: 0.9751 - val_loss: 0.0869
Epoch 7/10
117s 583ms/step - accuracy: 0.9490 - loss: 0.1888 - val_accuracy: 0.9729 - val_loss: 0.0933
Epoch 8/10
116s 581ms/step - accuracy: 0.9576 - loss: 0.1508 - val_accuracy: 0.9736 - val_loss: 0.0835
Epoch 9/10
115s 576ms/step - accuracy: 0.9637 - loss: 0.1220 - val_accuracy: 0.9751 - val_loss: 0.0755
Epoch 10/10
113s 566ms/step - accuracy: 0.9659 - loss: 0.1094 - val_accuracy: 0.9766 - val_loss: 0.0673
16s 375ms/step - accuracy: 0.9782 - loss: 0.0706
Test Loss: 0.0706
Test Accuracy: 0.9782



I first trained a baseline CNN from scratch, which achieved about 90% test accuracy. Then I implemented a transfer learning model using EfficientNetB0 pretrained on ImageNet. In the first stage, I froze the backbone and trained only the classification head, which already gave me very strong performance. In the second stage, I fine-tuned the upper layers with a smaller learning rate. The final model reached about 97.8% test accuracy, which was a significant improvement over the baseline.

This suggests that pretrained visual representations transfer very well to plant disease recognition, especially since the task depends heavily on texture and local visual patterns.

9. 对transfer learning model做加强版的evaluation（人工干扰）

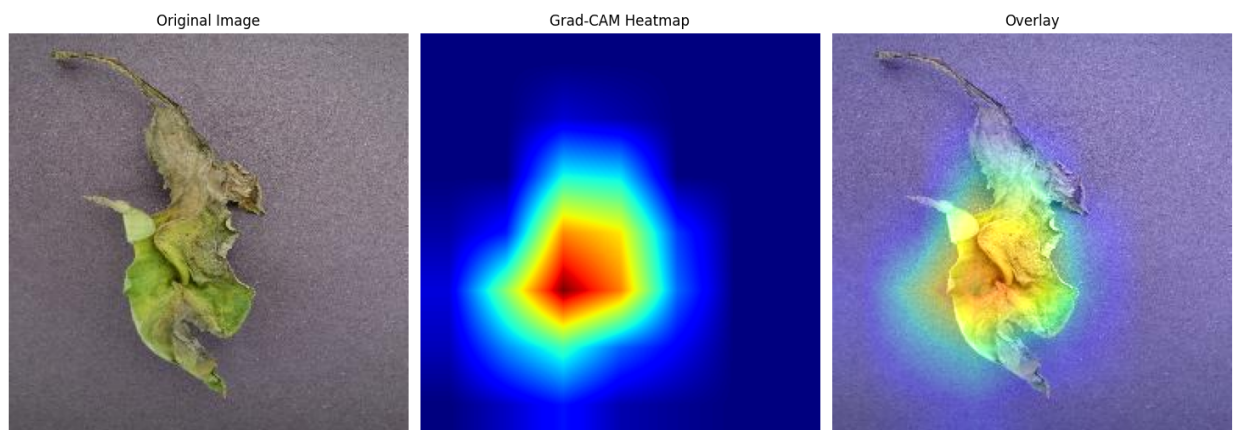
===== Summary =====

clean -> loss=0.0706, accuracy=0.9782
blur -> loss=0.2559, accuracy=0.9025
brightness -> loss=0.0760, accuracy=0.9760
rotation -> loss=0.1785, accuracy=0.9425
crop -> loss=0.0956, accuracy=0.9760
noise -> loss=0.2158, accuracy=0.9193
clutter -> loss=0.1885, accuracy=0.9353

Compared with my baseline CNN, the transfer learning model achieved significantly higher accuracy and was much more robust to different perturbations. For example, under blur and crop conditions, the baseline model dropped to around 70%, while the transfer model still maintained over 90% accuracy. Under background clutter, the improvement was even more significant, from about 63% to over 93%.

10. 加Grad-CAM

简述：在model最后一个convolution层，取出所有的feature map，和最后结果的这个分类做gradient，求出为了得到这个结果每个feature map的权重，最后将所有feature map乘以权重相加得到热力图，normalization这个热力图（删除负值），叠加到原始图片上



To improve interpretability, I used Grad-CAM to visualize which regions of the image contributed most to the model's prediction. This allowed me to verify whether the model was focusing on the diseased leaf regions rather than irrelevant background areas.